Machine Learning-Based Early Prediction and Classification of Chronic Kidney Disease Using Clinical Parameters

(CKD Prediction using Supervised Machine Learning)

Domain: Medical / Healthcare | Category: Supervised Machine Learning | Language: Python

Problem Statement:

Chronic Kidney Disease (CKD) is a progressive condition in which the kidneys gradually lose their ability to filter waste and excess fluid from the blood. It is frequently called a "silent" disease because patients often show no obvious symptoms until a large part of kidney function has already been lost. Early identification is therefore important, as timely treatment and lifestyle management can slow progression and reduce the risk of kidney failure. Conventional assessment depends on a clinician manually interpreting several laboratory and clinical readings together (such as serum creatinine, blood urea, haemoglobin, albumin, blood pressure and specific gravity), which is time-consuming, subjective and difficult to scale for large numbers of patients.

There is, therefore, a need for an automated, fast and accessible system that can analyse a patient's clinical parameters and classify the patient as CKD or Not CKD with a measurable probability, so that suspected cases can be flagged early for a nephrologist's review. This project designs and implements a web-based CKD classification system that preprocesses the UCI Chronic Kidney Disease dataset (imputation of missing values, label encoding and feature scaling), trains and compares five supervised classifiers, selects the best model, and evaluates it using Accuracy, Precision, Recall, F1-Score and the Confusion Matrix. The selected model is deployed through a Flask application that stores every prediction in an SQLite database. The system is an academic machine learning prototype and not a clinically validated diagnostic tool.

Objectives:

1. To collect the UCI Chronic Kidney Disease dataset (400 patient records, 24 clinical parameters) and load it for analysis.
2. To preprocess the clinical data by cleaning text values, converting numeric columns and encoding the target class (CKD / Not CKD).
3. To handle missing values using median imputation for numeric features and most-frequent imputation for categorical features, and to apply label encoding and standard scaling.
4. To analyse the clinical features through exploratory data analysis (class distribution, missing values, feature distributions, correlation heatmap and biomarker boxplots).
5. To develop and train five supervised machine learning models (Logistic Regression, Decision Tree, Random Forest, SVM and KNN) and select the best model using the F1-score.
6. To evaluate the models on an unseen test set and report Accuracy, Precision, Recall, F1-score, ROC-AUC, 5-fold cross-validation accuracy and the Confusion Matrix.

7. To build a Flask-based web application that accepts patient clinical parameters, displays the CKD prediction, and persists every prediction (patient ID, result, probability, timestamp) in an SQLite database as prediction history.

Software / Hardware Required:

- **Operating System:** Windows / Linux / macOS (standard personal computer; no GPU required)
- **IDE:** Visual Studio Code
- **Language:** Python 3.x
- **Backend Framework:** Flask (with Jinja2 HTML templates)
- **Machine Learning:** scikit-learn (LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, SVC, KNeighborsClassifier, SimpleImputer, LabelEncoder, StandardScaler)
- **Data Processing and Visualization:** Pandas, NumPy, Matplotlib, Seaborn
- **Model Persistence:** Joblib
- **Database:** SQLite3
- **Frontend:** HTML5, CSS3, JavaScript
- **Dataset:** UCI Chronic Kidney Disease dataset (400 records, 24 clinical features, CKD / Not CKD classes), downloaded using the ucimlrepo package

Project Directory Structure:

The project ("CKD Prediction System") is organized as a Flask application with separate folders for the dataset, trained models, static assets (CSS, JavaScript and generated charts) and HTML templates, as shown below. The scripts preprocess.py, train.py and eda.py handle preprocessing, model training and data analysis, while app.py runs the web application and the SQLite history database (ckd_history.db).

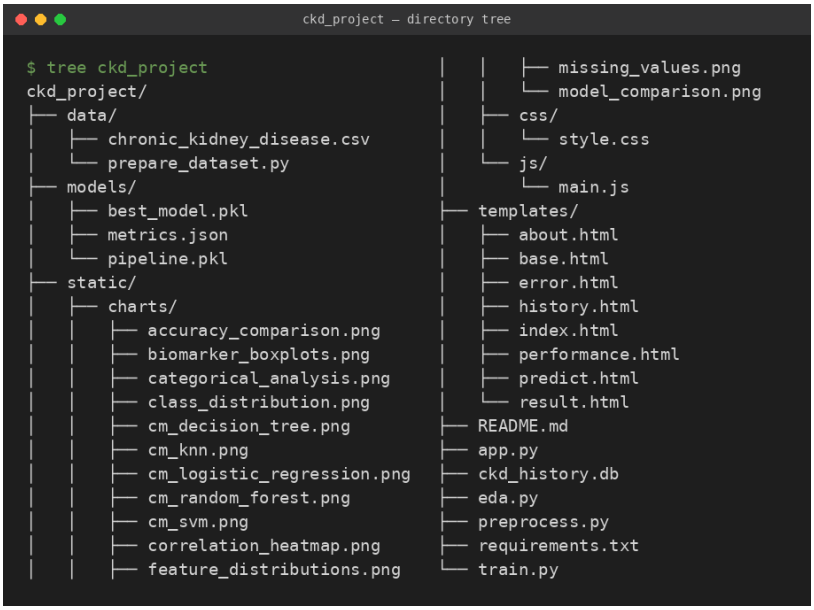


Fig 1: Project directory structure of CKD Prediction System (directory tree generated from the project folder)

Procedure:

1. Collect the UCI Chronic Kidney Disease dataset using data/prepare_dataset.py (ucimlrepo, dataset id 336) and save it as data/chronic_kidney_disease.csv (400 records, 24 clinical features and the class column).
2. Load the CSV into a Pandas DataFrame in preprocess.py (load_dataset), strip whitespace, convert text to lower case and replace "?" and empty strings with NaN.
3. Clean the data (clean_and_prepare): keep the 24 features, convert the 14 numeric columns using pd.to_numeric, clean the 10 categorical columns and encode the target as ckd = 1 and notckd = 0.
4. Perform exploratory data analysis with eda.py and save the class distribution, missing-value, feature-distribution, correlation-heatmap, boxplot and categorical-analysis charts in static/charts/.
5. Split the data into 80% training (320 records) and 20% testing (80 records) using a stratified train_test_split with random_state = 42.
6. Handle missing values (1012 missing cells in the dataset) using SimpleImputer with the median strategy for numeric features and the most-frequent strategy for categorical features, fitted on the training data only.
7. Encode the 10 categorical features using LabelEncoder and scale the 14 numeric features using StandardScaler; save the fitted preprocessing objects as models/pipeline.pkl.
8. Train five classifiers in train.py: Logistic Regression, Decision Tree (max_depth = 8), Random Forest (100 trees), SVM (RBF kernel) and KNN (k = 5).
9. Evaluate every model on the test set using Accuracy, Precision, Recall, F1-Score, ROC-AUC, 5-fold cross-validation accuracy and the Confusion Matrix, and compare the results.
10. Select the best model by F1-score (Random Forest), then save it as models/best_model.pkl and the metrics as models/metrics.json.
11. Launch the Flask application using python app.py (http://127.0.0.1:5000); enter the patient's clinical parameters in the Predict CKD form, which are validated, preprocessed with the saved pipeline and classified as CKD or Not CKD with the CKD probability.
12. Store every prediction (patient ID, result, probability, timestamp) in the SQLite table "predictions" of ckd_history.db and view the stored records on the Prediction History page.

Implementation Code:

Representative implementation of the key modules used in the project, based on the pipeline shown in the application (file names match the project's source files).

1. Dataset Loading and Preprocessing (preprocess.py)

```
import os
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
import joblib

DATA_DIR   = os.path.join(os.path.dirname(__file__), 'data')
MODELS_DIR = os.path.join(os.path.dirname(__file__), 'models')

NUMERIC_FEATURES =
['age','bp','sg','al','su','bgr','bu','sc','sod','pot','hemo','pcv','wbcc','rbcc']
CATEG_FEATURES   = ['rbc','pc','pcc','ba','htn','dm','cad','appet','pe','ane']
ALL_FEATURES     = NUMERIC_FEATURES + CATEG_FEATURES
TARGET           = 'class'

def load_dataset():
    """Load CSV, strip whitespace, return raw DataFrame."""
    path = os.path.join(DATA_DIR, 'chronic_kidney_disease.csv')
    df = pd.read_csv(path)
    # Strip whitespace from string columns
    for col in df.columns:
        if df[col].dtype == object:
            df[col] = df[col].str.strip().str.lower().replace({'?': np.nan, '': np.nan})
    return df

def clean_and_prepare(df):
    """
    1. Keep only needed columns
    2. Convert numeric cols to float
    3. Encode target: ckd=1, notckd=0
    """
    df = df[ALL_FEATURES + [TARGET]].copy()
    # Convert numeric columns
    for col in NUMERIC FEATURES:
        df[col] = pd.to_numeric(df[col], errors='coerce')
    # Clean categoricals
    for col in CATEG FEATURES:
        df[col] = df[col].astype(str).str.strip().str.lower()
        df[col] = df[col].replace({'nan': np.nan, '?': np.nan})
    # Encode target
    df[TARGET] = df[TARGET].str.strip().str.lower()
    df[TARGET] = df[TARGET].map({'ckd': 1, 'notckd': 0})
    return df

def get_train_test_split():
    """Full preprocessing pipeline returning X_train, X_test, y_train, y_test, pipeline."""
    df  = load_dataset()
    df  = clean_and_prepare(df)

    # Drop rows where target is null
    df  = df.dropna(subset=[TARGET])

    X   = df[ALL_FEATURES]
    y   = df[TARGET].astype(int)

    # Stratified split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )
```

```
    # Build pipeline on TRAIN data only (no data leakage)
    pipeline = build_pipeline(X_train)

    X_train_t = apply_pipeline(X_train, pipeline)
    X_test_t  = apply_pipeline(X_test, pipeline)

    return X_train_t, X_test_t, y_train.values, y_test.values, pipeline, df
```

2. Missing Value Handling and Feature Encoding (preprocess.py)

```
def build_pipeline(X_train):
    """
    Fit imputers, label encoders, and scaler on training data only.
    Returns fitted objects.
    """
    # Numeric imputer - median strategy
    num_imputer = SimpleImputer(strategy='median')
    num_imputer.fit(X_train[NUMERIC_FEATURES])

    # Categorical imputer - most frequent
    cat_imputer = SimpleImputer(strategy='most_frequent')
    cat_imputer.fit(X_train[CATEG_FEATURES])

    # Label encoders for each categorical column
    label_encoders = {}
    X_cat_filled = cat_imputer.transform(X_train[CATEG_FEATURES])
    for i, col in enumerate(CATEG_FEATURES):
        le = LabelEncoder()
        le.fit(X_cat_filled[:, i])
        label_encoders[col] = le

    # Standard scaler on numeric features
    X_num_filled = num_imputer.transform(X_train[NUMERIC_FEATURES])
    scaler = StandardScaler()
    scaler.fit(X_num_filled)

    pipeline = {
        'num_imputer':   num_imputer,
        'cat_imputer':   cat_imputer,
        'label_encoders': label_encoders,
        'scaler':        scaler,
        'numeric_features': NUMERIC_FEATURES,
        'categ_features':   CATEG_FEATURES,
        'all_features':     ALL_FEATURES,
    }
    return pipeline

def apply_pipeline(X, pipeline):
    """Transform features using a fitted pipeline."""
    num_imp    = pipeline['num_imputer']
    cat_imp    = pipeline['cat_imputer']
    les        = pipeline['label encoders']
    scaler     = pipeline['scaler']
    num_feats  = pipeline['numeric_features']
    cat_feats  = pipeline['categ features']

    # Impute
    X_num = num_imp.transform(X[num_feats])
```

```
    X_cat = cat_imp.transform(X[cat_feats])

    # Encode categoricals
    X_cat_enc = np.zeros_like(X_cat, dtype=float)
    for i, col in enumerate(cat_feats):
        le = les[col]
        col_vals = X_cat[:, i]
        # Handle unseen labels gracefully
        encoded = []
        for v in col_vals:
            if v in le.classes_:
                encoded.append(le.transform([v])[0])
            else:
                encoded.append(0)
        X_cat_enc[:, i] = encoded

    # Scale numerics
    X_num_scaled = scaler.transform(X_num)

    # Concatenate
    X_transformed = np.hstack([X_num_scaled, X_cat_enc])
    return X_transformed
```

3. Machine Learning Model Training (train.py)

```
import joblib

from sklearn.linear_model    import LogisticRegression
from sklearn.tree            import DecisionTreeClassifier
from sklearn.ensemble        import RandomForestClassifier
from sklearn.svm             import SVC
from sklearn.neighbors       import KNeighborsClassifier
from sklearn.metrics         import (accuracy_score, precision_score,
                                     recall_score, f1_score,
                                     confusion_matrix, roc_auc_score,
                                     classification_report)
from sklearn.model_selection import cross_val_score

CLASSIFIERS = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree':       DecisionTreeClassifier(max_depth=8, random_state=42),
    'Random Forest':       RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM':                 SVC(kernel='rbf', probability=True, random_state=42),
    'KNN':                 KNeighborsClassifier(n_neighbors=5),
}

# Inside main(): train every classifier, then select the best by F1-score
    results = []
    for name, clf in CLASSIFIERS.items():
        res = evaluate model(name, clf, X tr, X te, y tr, y te)
        results.append(res)

    # Select best model by F1-score (best for medical classification)
    best = max(results, key=lambda r: r['f1'])

    # Save best model
    joblib.dump(best['classifier'], os.path.join(MODELS_DIR, 'best_model.pkl'))
    joblib.dump(pipeline,          os.path.join(MODELS_DIR, 'pipeline.pkl'))
```

4. Model Evaluation (train.py)

```
from sklearn.metrics         import (accuracy_score, precision_score,
                                     recall_score, f1_score,
                                     confusion_matrix, roc_auc_score,
                                     classification_report)

def evaluate_model(name, clf, X_tr, X_te, y_tr, y_te):
    clf.fit(X_tr, y_tr)
    y_pred   = clf.predict(X_te)
    y_proba  = clf.predict_proba(X_te)[:, 1] if hasattr(clf, 'predict_proba') else None

    acc  = accuracy_score(y_te, y_pred)
    prec = precision_score(y_te, y_pred, zero_division=0)
    rec  = recall_score(y_te, y_pred, zero_division=0)
    f1   = f1_score(y_te, y_pred, zero_division=0)
    cm   = confusion_matrix(y_te, y_pred)
    auc  = roc_auc_score(y_te, y_proba) if y_proba is not None else None

    cv_scores = cross_val_score(clf, X_tr, y_tr, cv=5, scoring='accuracy')

    print(f"\n--- {name} ---")
    print(f"  Accuracy : {acc:.4f}")
    print(f"  Precision: {prec:.4f}")
    print(f"  Recall   : {rec:.4f}")
    print(f"  F1-Score : {f1:.4f}")
    if auc: print(f"  ROC-AUC  : {auc:.4f}")
    print(f"  CV (5-fold) Accuracy: {cv_scores.mean():.4f} ± {cv_scores.std():.4f}")
    print(classification_report(y_te, y_pred, target_names=['Not CKD','CKD']))

    return {
        'name':      name,
        'classifier': clf,
        'accuracy':  acc,
        'precision': prec,
        'recall':    rec,
        'f1':        f1,
        'auc':       auc,
        'cv_mean':   float(cv_scores.mean()),
        'cv_std':    float(cv_scores.std()),
        'cm':        cm,
    }
```

5. Patient Prediction Function and Flask Route (app.py)

```
def predict_ckd(data):
    """Run model prediction on parsed form data."""
    import pandas as pd
    from preprocess import apply_pipeline

    df = pd.DataFrame([data], columns=FEATURE_ORDER)
    X  = apply_pipeline(df, PIPELINE)

    pred     = MODEL.predict(X)[0]
    proba    = MODEL.predict_proba(X)[0] if hasattr(MODEL, 'predict_proba') else None
    prob_ckd = float(proba[1]) if proba is not None else None
```

```
    label = 'CKD' if pred == 1 else 'Not CKD'
    return label, prob_ckd

@app.route('/predict', methods=['GET', 'POST'])
def predict():
    if request.method == 'GET':
        return render_template('predict.html', feature_labels=FEATURE_LABELS)

    data, errors = parse_input(request.form)
    if errors:
        return render_template('predict.html', feature_labels=FEATURE_LABELS,
                               errors=errors, form_data=request.form)

    try:
        label, prob = predict_ckd(data)
    except Exception as e:
        return render_template('predict.html', feature_labels=FEATURE_LABELS,
                               errors=[f"Prediction error: {e}"], form_data=request.form)

    patient_id = f"PAT-{datetime.now().strftime('%Y%m%d%H%M%S')}"
    save_prediction(patient_id, label, prob)

    prob_pct = round(prob * 100, 1) if prob is not None else None
    return render_template('result.html', label=label, prob=prob,
                           prob_pct=prob_pct, patient_id=patient_id,
                           form data=data)
```

6. SQLite Prediction History (app.py)

```
def init_db():
    conn = sqlite3.connect(DB_PATH)
    c    = conn.cursor()
    c.execute('''
        CREATE TABLE IF NOT EXISTS predictions (
            id          INTEGER PRIMARY KEY AUTOINCREMENT,
            patient_id  TEXT,
            prediction  TEXT,
            probability REAL,
            timestamp   TEXT
        )
    ''')
    conn.commit()
    conn.close()

def save_prediction(patient_id, prediction, probability):
    conn = sqlite3.connect(DB_PATH)
    c    = conn.cursor()
    c.execute(
        'INSERT INTO predictions (patient_id, prediction, probability, timestamp) VALUES
(?,?,?,?)',
        (patient_id, prediction, probability, datetime.now().strftime('%Y-%m-%d %H:%M:%S'))
    )
    conn.commit()
    conn.close()

def get history(limit=50):
    conn  = sqlite3.connect(DB_PATH)
    c     = conn.cursor()
    c.execute('SELECT id, patient id, prediction, probability, timestamp FROM predictions ORDER
BY id DESC LIMIT ?', (limit,))
    rows  = c.fetchall()
    conn.close()
    return rows

@app.route('/history')
def history():
    rows = get_history(100)
```

```
    return render_template('history.html', rows=rows)
```

Output (Step-by-Step Screenshots):

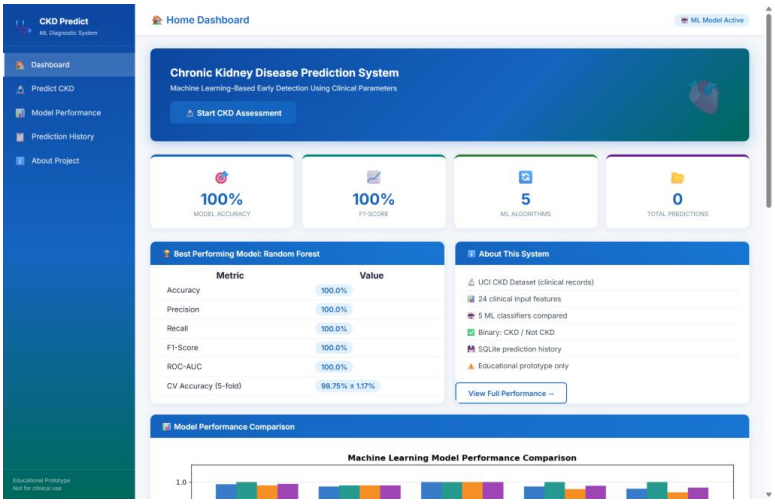


Fig 2: Step 1 — Home dashboard of the CKD Prediction System showing the active model, ML algorithms and best-model metrics

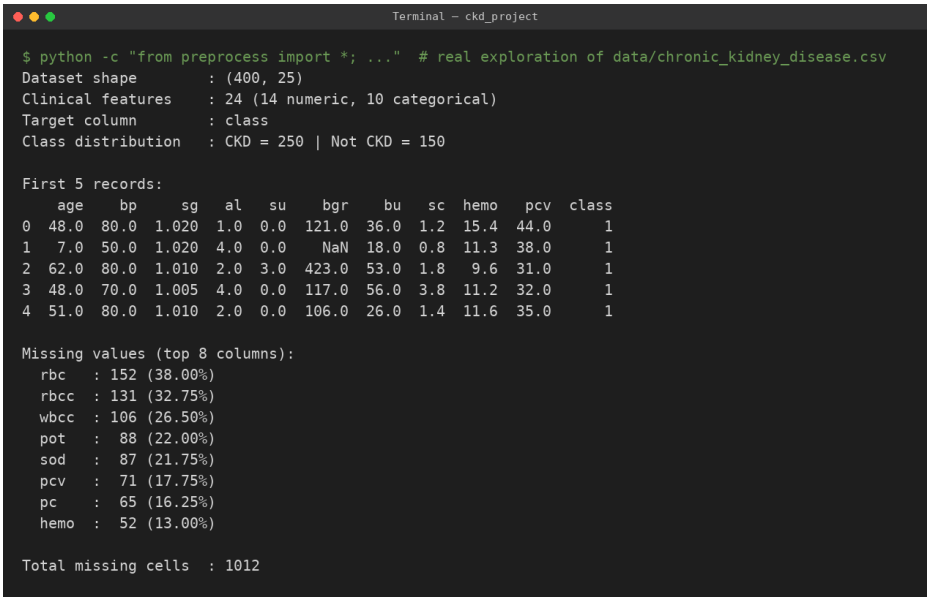


Fig 3: Step 2 — Dataset loading and clinical data exploration (400 records, 24 features, class distribution and missing values)

```
Terminal — ckd_project

$ python preprocess.py
Dataset shape: (400, 25)
Train: (320, 24), Test: (80, 24)
Class balance (train): {np.int64(0): np.int64(120), np.int64(1): np.int64(200)}
Pipeline saved.

$ # missing values before/after imputation (project pipeline)
Missing before imputation: 1012
Missing after  imputation (train): 0 | (test): 0
Imputers  : numeric -> median | categorical -> most_frequent
Encoding  : LabelEncoder on 10 categorical features
Scaling   : StandardScaler on 14 numeric features
Train mean of scaled numeric cols (first 3): [0. 0. 0.]
Label classes (rbc): ['abnormal', 'normal']
```

Fig 4: Step 3 — Data preprocessing and missing value handling (stratified 320/80 split; median and most-frequent imputation, label encoding, standard scaling)

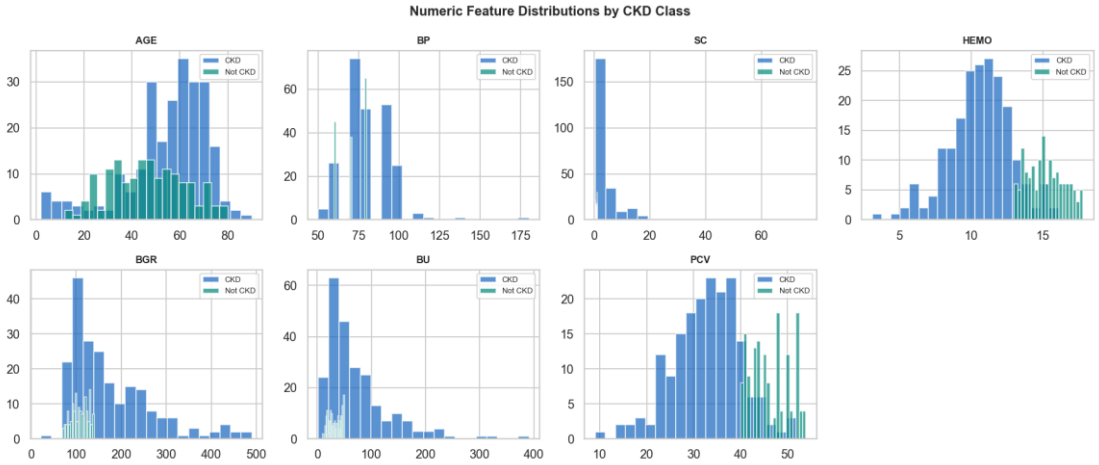


Fig 5: Step 4 — Exploratory data analysis: distributions of the clinical features (generated by eda.py)

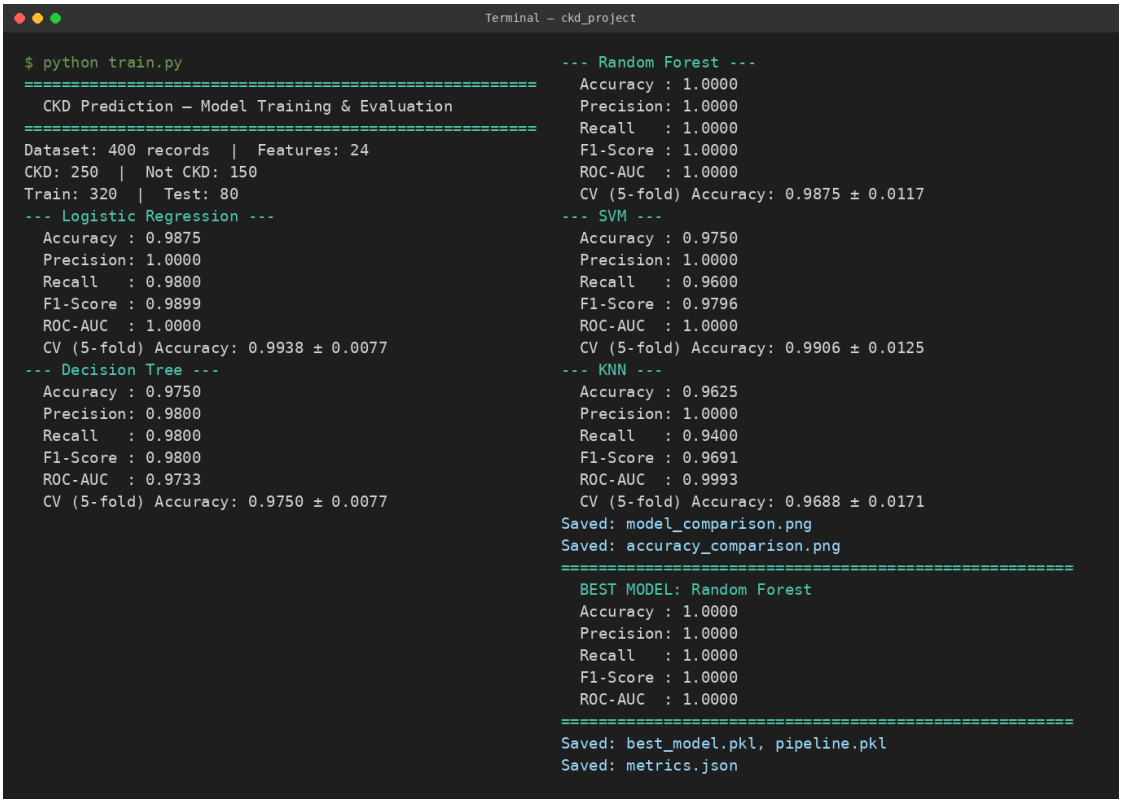


Fig 6: Step 5 — Machine learning model training and evaluation output of train.py for the five classifiers



Fig 7: Step 6 — Model performance comparison of the five classifiers (Model Performance page)

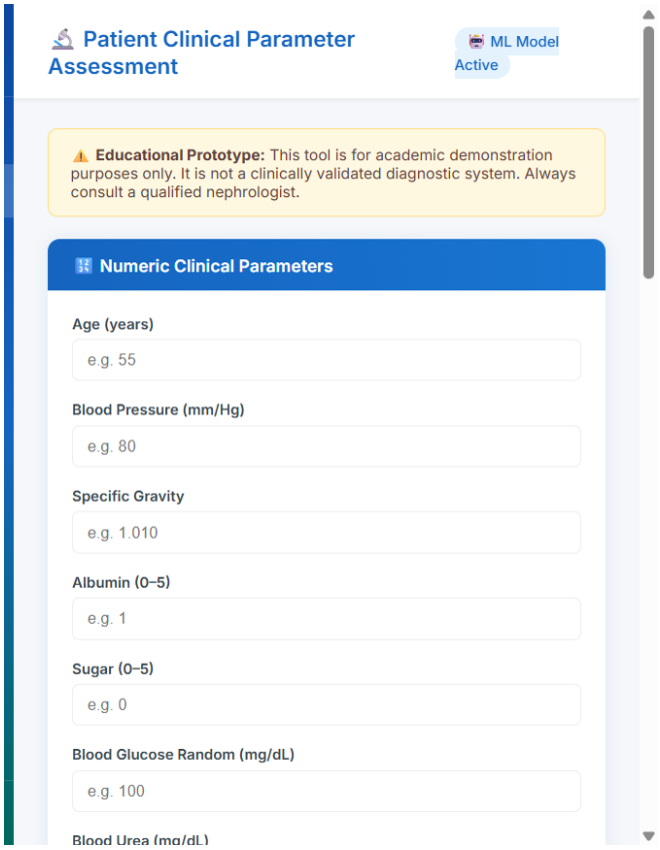


Fig 8: Step 7 — Patient clinical parameter input form (Predict CKD page)

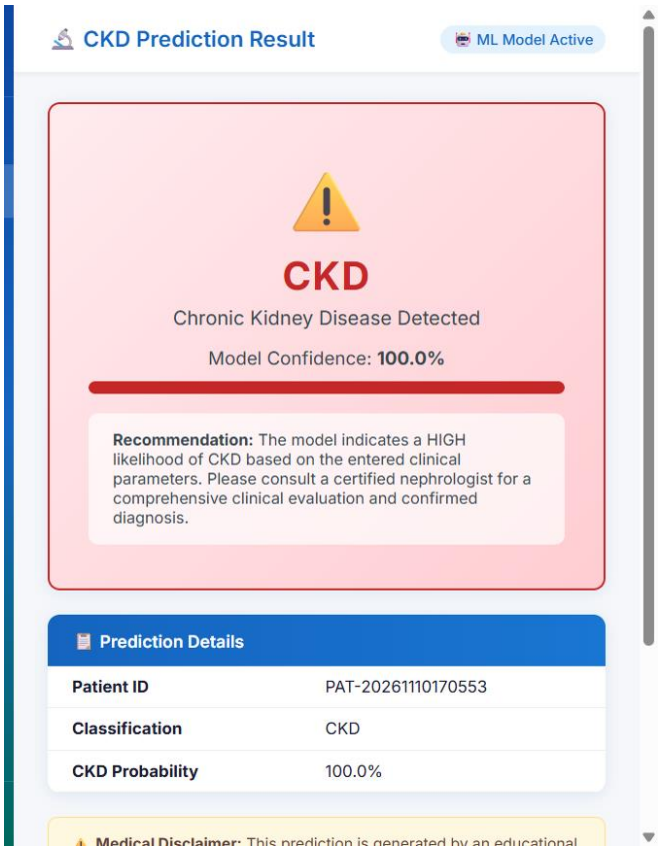


Fig 9: Step 8 — Prediction result: CKD detected, with the CKD probability and patient ID

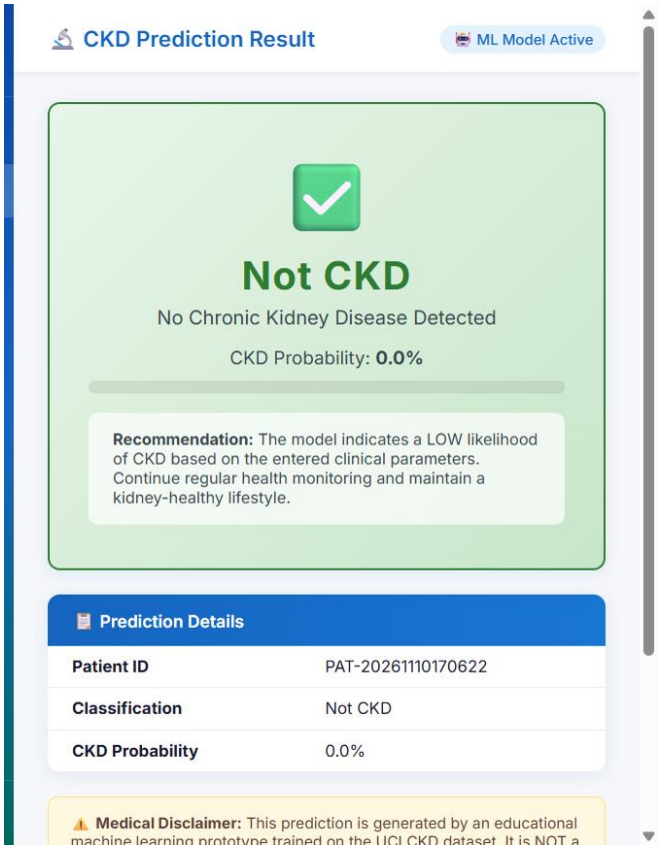


Fig 10: Step 8 — Prediction result: Not CKD, with the CKD probability and patient ID

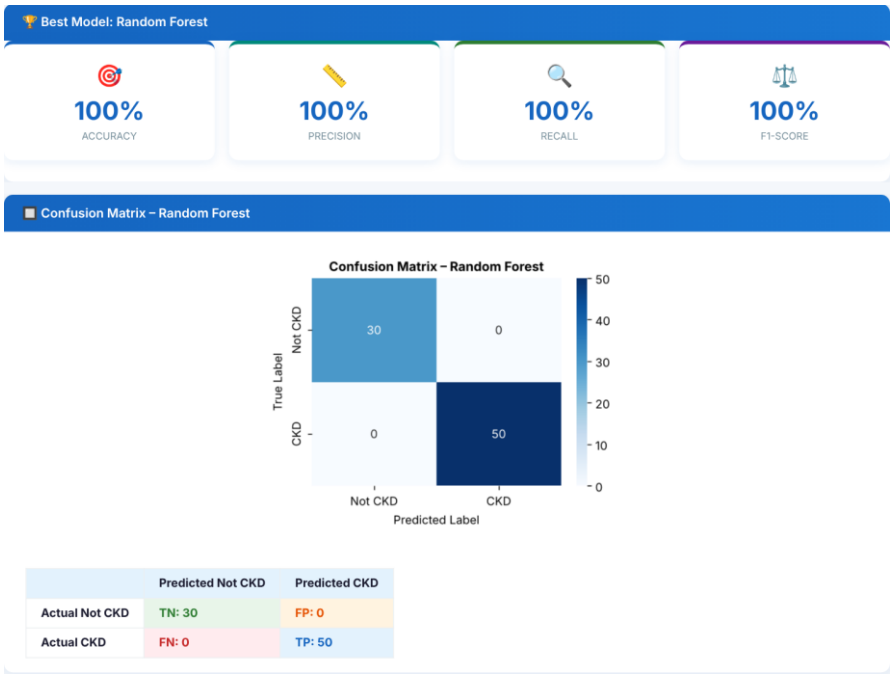


Fig 11: Step 9 — Performance evaluation metrics and confusion matrix of the selected Random Forest model

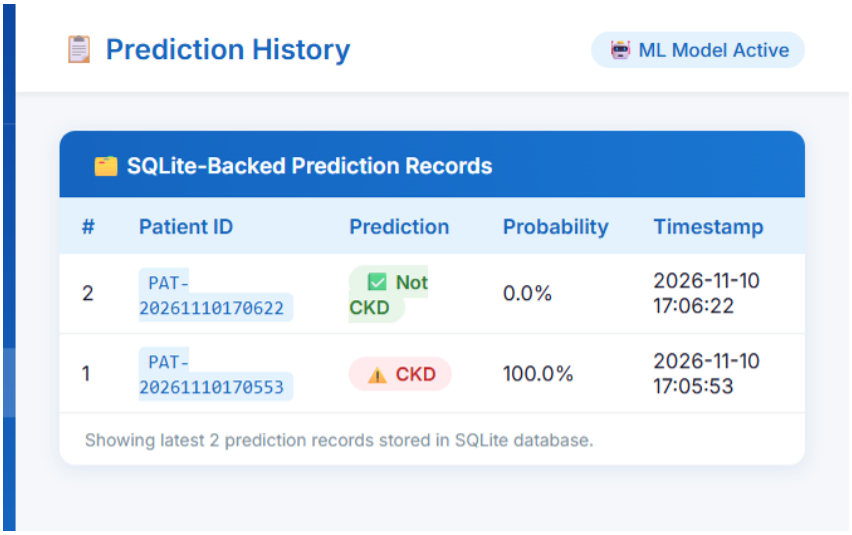


Fig 12: Step 10 — Prediction history stored in the SQLite database (ckd_history.db)

Result:

A web-based Chronic Kidney Disease prediction system was successfully designed and implemented using Python, scikit-learn, Flask and SQLite. The UCI CKD dataset (400 records, 24 clinical features; 250 CKD and 150 Not CKD) was preprocessed using median and most-frequent imputation, label encoding and standard scaling, and split 80:20 (320 training, 80 testing records). Five classifiers (Logistic Regression, Decision Tree, Random Forest, SVM and KNN) were trained and compared, and the Random Forest model was selected as the best model based on F1-score. On the held-out test set it achieved an Accuracy of 100.00%, Precision of 100.00%, Recall of 100.00% and F1-Score of 100.00% (ROC-AUC 1.0000), with the confusion matrix TN = 30, FP = 0, FN = 0, TP = 50. Its 5-fold cross-validation accuracy on the training data was 98.75% ± 1.17%. The results of all five models are given below.

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	CV Acc. (5-fold)
Logistic Regression	98.75%	100.00%	98.00%	98.99%	1.0000	99.38% ± 0.77
Decision Tree	97.50%	98.00%	98.00%	98.00%	0.9733	97.50% ± 0.77
Random Forest (selected)	**100.00%**	**100.00%**	**100.00%**	**100.00%**	**1.0000**	**98.75% ± 1.17**
SVM	97.50%	100.00%	96.00%	97.96%	1.0000	99.06% ± 1.25
KNN	96.25%	100.00%	94.00%	96.91%	0.9993	96.88% ± 1.71

The Flask application accepts the patient's clinical parameters, applies the saved preprocessing pipeline, returns the CKD / Not CKD classification with the CKD probability, and records every prediction (patient ID, result, probability and timestamp) in the SQLite database for the prediction history. The perfect test-set scores should be interpreted with caution, because the test set contains only 80 records and the cross-validation accuracy (98.75%) is slightly lower.

This project is an academic machine learning prototype and is not a clinically validated diagnostic system; it must not be used for actual medical diagnosis or treatment decisions.